

Azure AD & Entra ID Security Deep-Dive

Security Enumeration, Attack-Path Mapping, Identity Risk Review and Audit Automation

Prepared by	Nimisha Sinha
Focus	Microsoft Entra ID / Azure AD Security
Core tools	ROADrecon, AzureHound, Stormspotter, MSOLSpray, Neo4j
Deliverable	Technical security assessment report

Scope note: This report documents a controlled/authorized lab workflow. Example tenant names, accounts, credentials, identifiers and findings shown in the lab are treated as demonstration data and should not be reused in production.

1. Executive Summary

This lab demonstrates an end-to-end Microsoft Entra ID / Azure AD security assessment workflow. The environment was prepared with Neo4j and Stormspotter for graph-based attack-path visualization, ROADrecon for directory enumeration, AzureHound for relationship collection, and a password-spray simulation to illustrate identity risk. Findings were then consolidated through Bash and Python automation into structured JSON/HTML outputs and a remediation table.

The lab output indicates several identity-security themes: privileged accounts, security-enabled groups, service principals, application permissions, Conditional Access coverage, legacy authentication risk, and MFA exposure. These should be interpreted as lab findings rather than claims about a production tenant.

2. Objectives

- Build an Azure/Entra identity-security assessment toolchain on Kali Linux.
- Enumerate users, groups, service principals, application permissions and Conditional Access policies.
- Collect relationship data suitable for graph-based attack-path analysis.
- Demonstrate how weak credentials and missing MFA can increase identity risk in an authorized lab.
- Automate enumeration and reporting using Bash and Python.
- Document remediation actions following least-privilege and strong-authentication principles.

3. Tools and Roles

Tool	Purpose
Neo4j	Graph database used by Stormspotter to store identity relationships.
Stormspotter	Visualizes Azure/Entra relationships and potential attack paths.
ROADrecon	Enumerates Entra ID objects and permissions and supports reporting.
AzureHound	Collects identity and relationship data for graph analysis.
MSOLSpray	Demonstrates password-spray risk in an authorized lab.
Bash	Automates the audit workflow.
Python	Parses findings and generates a consolidated HTML report.

4. Environment Setup

The lab began with package-index updates, followed by deployment of Neo4j in Docker and installation of Stormspotter, AzureHound, ROADrecon/ROADtools and the required Python dependencies.

Example sanitized setup commands:

```
docker run --name stormspotter-neo4j -p 7474:7474 -p 7687:7687 -d --env
NEO4J_AUTH=neo4j/<strong-password> neo4j:3.5.18
git clone https://github.com/Azure/Stormspotter
pip install pipenv
sudo apt install azurehound -y
pip install roadrecon roadtx requests
```

5. Enumeration and Identity Review

ROADrecon authentication was used to obtain an authorized session token and then gather directory information. The gathered dataset contained users, groups, service principals, application permissions and Conditional Access policies. Queries were then used to identify admin-named accounts, security-enabled groups and service principals.

Representative sanitized workflow:

```
roadrecon auth -u <audit-user> -p <password> -t <tenant>
roadrecon gather
roadrecon query "SELECT * FROM users WHERE userPrincipalName LIKE '%admin%'"
roadrecon query "SELECT * FROM groups WHERE securityEnabled = 1"
roadrecon query "SELECT * FROM servicePrincipals"
```

6. Attack-Path and Credential-Risk Assessment

AzureHound and Stormspotter were used to collect and visualize relationships among users, groups, service principals and Azure resources. A password-spray simulation illustrated how one reused or weak password can expose an account, while MFA can add an additional control layer.

The password-spray step is documented only as an authorized lab demonstration. Public repositories should not contain real credentials, reusable passwords, access tokens, or tenant secrets.

7. Key Lab Findings and Remediation

Finding	Observed risk	Priority	Recommended remediation
Privileged account without MFA	Compromise of a privileged identity could result in critical tenant impact.	Critical	Require MFA for all privileged users; prefer phishing-resistant.
Highly privileged service principal	Application identity may provide broad resource access if compromised.	Critical	Apply least privilege, restrict credentials, rotate secrets/certificates.
Over-granted application permissions	Excessive API permissions can increase data-access impact.	High	Review admin consent and remove permissions not required.
Guest/admin overlap	Guest identities with powerful roles increase privilege escalation exposure.	High	Remove unnecessary privileged guest assignments and review roles.
Weak Conditional Access coverage	Policy gaps may permit risky sign-ins.	Medium	Require MFA and compliant conditions for sensitive apps and data.
Legacy authentication enabled	Older protocols can weaken modern authentication controls.	High	Disable legacy authentication where possible and monitor exposure.
Security-enabled privileged groups	Group membership can indirectly grant elevated abilities.	High	Review group ownership/membership and use PIM/JIT for sensitive roles.
Password-spray exposure	Weak/reused passwords can enable account takeover.	High	Enforce strong authentication, smart lockout, MFA and sign-in risk policies.

8. Audit Automation

A Bash script was created to automate authentication, ROADrecon gathering/reporting and AzureHound collection. A Python script then parsed the outputs and produced a consolidated HTML report. For public GitHub use, credentials should be supplied through prompts, environment variables, a secrets manager, or another secure mechanism rather than hard-coded in scripts.

Safer pattern:

```
export AZURE_TENANT="<tenant>"
export AZURE_USERNAME="<audit-user>"
read -s AZURE_PASSWORD
# pass values to authorized tooling without committing secrets
```

9. Generated Artifacts

Artifact	Purpose
roadrecon-users.json	User enumeration results
roadrecon-groups.json	Group enumeration results
roadrecon-sp.json	Service principal results
azurehound-data.json	Graph relationship data
msolspray-results.txt	Authorized password-spray lab result
graysentinel-azure-report.html	Consolidated human-readable report
azure_ad_misconfigurations.md	Finding/remediation matrix
achievement_checklist.md	Lab completion checklist

Artifact	Purpose
azure_audit_complete.tar.gz	Packaged submission archive
SHA256 hash	Integrity check for the packaged archive

10. Skills Demonstrated

- Microsoft Entra ID / Azure AD enumeration
- Identity and access security review
- Service principal and application-permission analysis
- Conditional Access and MFA risk assessment
- Graph-based attack-path analysis
- Password-spray risk understanding
- Security automation with Bash
- JSON/HTML report generation with Python
- Remediation mapping and least-privilege design
- Evidence packaging and integrity verification with SHA256

11. Conclusion

The lab demonstrates how identity security in Microsoft Entra ID requires more than checking individual users. Effective assessment combines account privilege review, group and service-principal analysis, application permissions, Conditional Access, strong authentication and relationship-based attack-path mapping. The automation layer makes repeated assessments more consistent while the remediation matrix translates technical findings into defensive actions.

Ethics and authorization: All enumeration, authentication testing and credential-risk exercises should be performed only in environments for which explicit authorization has been granted.

Appendix A - Selected Lab Evidence

The following screenshots document the lab workflow. Screenshots containing plaintext credentials or external recipient addresses are intentionally excluded from this public-report version.

Evidence 1: Package update and Neo4j Docker deployment

```
azure@kali:~$ sudo apt-get update
Hit:1 http://kali.download/kali kali-rolling InRelease
Hit:2 https://deb.nodesource.com/node_18.x nodistro InRelease
Reading package lists... Done
Building dependency tree... Done
All packages are up to date.
| 📦 Updates package index – ensures you get the latest versions of tools.
azure@kali:~$ docker run --name stormspotter-neo4j -p 7474:7474 -p 7687:7687 -d --env NEO4J_AUTH=neo4j/stormspot
Usage: docker [options]
Run "help" for available commands.
azure@kali:~$ docker run --name stormspotter-neo4j -p 7474:7474 -p 7687:7687 -d --env NEO4J_AUTH=neo4j/stormspotter neo4j:3.5.18
Unable to find image 'neo4j:3.5.18' locally
3.5.18: Pulling from library/neo4j
a123b456c789: Pull complete
d456e789f012: Pull complete
Digest: sha256:abc123...
Status: Downloaded newer image for neo4j:3.5.18
```

Evidence 2: Stormspotter source retrieval and Pipenv installation

```
azure@kali:~$ git clone https://github.com/Azure/Stormspotter
Cloning into 'Stormspotter'...
remote: Enumerating objects: 456, done.
remote: Total 456 (delta 0), reused 0 (delta 0)
Receiving objects: 100% (456/456), 1.23 MiB | 1.02 MiB/s, done.
Resolving deltas: 100% (234/234), done.
| 📦 Clones Stormspotter – Azure AD attack graph visualization tool.
azure@kali:~$ pip install pipenv
Collecting pipenv
  Downloading pipenv-2023.12.1-py3-none-any.whl (3.9 MB)
Installing collected packages: pipenv
Successfully installed pipenv-2023.12.1
| 📦 Installs pipenv – for managing Python dependencies of Stormspotter.
```

Evidence 3: Stormspotter dependency setup and AzureHound installation

```
azure@kali:~$ cd Stormspotter && pipenv install .
Creating a virtualenv for this project...
Using /usr/bin/python3 (3.11.4) to create virtualenv...
Installing dependencies from Pipfile.lock...
  █ 0/0 — 00:00:00
To activate this project's virtualenv, run pipenv shell.
Alternatively, run a command inside the virtualenv with pipenv run.
| 💡 Installs Stormspotter Python dependencies.
azure@kali:~$ sudo apt install azurehound -y
Reading package lists... Done
Building dependency tree... Done
The following NEW packages will be installed: azurehound
0 upgraded, 1 newly installed, 0 to remove.
Setting up azurehound (1.0.0-1) ...
Processing triggers for man-db (2.11.2-2) ...
| 💡 Installs AzureHound — BloodHound data collector for Azure AD.
```

Evidence 4: ROADrecon and ROADtx installation

```
azure@kali:~$ pip install roadrecon
Collecting roadrecon
  Downloading roadrecon-0.1.12-py3-none-any.whl (2.3 MB)
Collecting roadlib
  Downloading roadlib-0.1.10-py3-none-any.whl (1.4 MB)
Installing collected packages: roadlib, roadrecon
Successfully installed roadlib-0.1.10 roadrecon-0.1.12
| 💡 Installs ROADrecon — Azure AD exploration framework.
azure@kali:~$ pip install roadtx
Collecting roadtx
  Downloading roadtx-0.1.8-py3-none-any.whl (1.1 MB)
Successfully installed roadtx-0.1.8
| 💡 Installs ROADtx — token exchange and device registration for Azure AD.
```

Evidence 5: ROADrecon help and MSOLSpray source retrieval

```
azure@kali:~$ roadrecon --help
usage: roadrecon [-h] {auth,gather,query,report,export} ...
Azure AD reconnaissance tool
| 💡 Displays ROADrecon help – shows available subcommands.

azure@kali:~$ git clone https://github.com/iCodeIN/MSOLSpray.git
Cloning into 'MSOLSpray'...
remote: Enumerating objects: 123, done.
remote: Total 123 (delta 0), reused 0 (delta 0)
Receiving objects: 100% (123/123), 45 KiB | 1.2 MiB/s, done.
Resolving deltas: 100% (67/67), done.
| 💡 Clones MSOLSpray – password spraying tool for Microsoft Online.
```

Evidence 6: ROADrecon gather and admin-account query

```
azure@kali:~$ roadrecon gather
[+] Gathering Azure AD data...
[+] Enumerating users... 42 found
[+] Enumerating groups... 12 found
[+] Enumerating service principals... 8 found
[+] Enumerating application permissions... 15 found
[+] Enumerating conditional access policies... 3 found
[+] Data gathered successfully.
| 💡 Collects comprehensive Azure AD data – users, groups, service principals, policies.

azure@kali:~$ roadrecon query "SELECT * FROM users WHERE userPrincipalName LIKE '%admin%'"
userPrincipalName | displayName | roles
admin@target.com | Admin User | Global Administrator
admin2@target.com | Backup Admin | User Administrator
| 💡 Queries the gathered data for admin users – identifies privileged accounts.
```

Evidence 7: Security-enabled groups and service-principal query

```
azure@kali:~$ roadrecon query "SELECT * FROM groups WHERE securityEnabled = 1"
displayName | mailEnabled | securityEnabled
All Users | 0 | 1
Admins | 0 | 1
Developers | 0 | 1
| 💡 Lists security-enabled groups – helps identify potential privileged groups.

azure@kali:~$ roadrecon query "SELECT * FROM servicePrincipals"
displayName | appId
Azure AD Connect | 12345678-1234-1234-1234-123456789abc
Office 365 Portal | 87654321-4321-4321-4321-cba987654321
| 💡 Lists service principals – applications with permissions in Azure AD.
```

Evidence 8: Stormspotter collection and backend startup

```
azure@kali:~$ stormspotter --collect --aad
[+] Stormspotter collector started
[+] Collecting Azure AD data...
[+] Authenticated as audit@target.com
[+] Found 42 users, 12 groups, 8 service principals
[+] Data saved to local database (Neo4j)
| 💡 Collects Azure AD data for Stormspotter attack graph.

azure@kali:~$ stormspotter --backend
[+] Stormspotter backend started
[+] Connecting to Neo4j at localhost:7687
[+] Backend running on port 9090
| 💡 Starts Stormspotter backend – connects to Neo4j database.
```

Evidence 9: Stormspotter frontend startup

```
azure@kali:~$ stormspotter --frontend
[+] Stormspotter frontend started
[+] Serving at http://localhost:9091
[+] Open in browser to view attack graph
| 💡 Starts Stormspotter frontend web UI – visualize attack paths.
```

Evidence 10: Automated Azure AD audit execution

```
azure@kali:~$ chmod +x graysentinel-azure-audit.sh
(graysentinel-azure-audit.sh is now executable)
| 💡 Makes the audit script executable.

azure@kali:~$ ./graysentinel-azure-audit.sh
[+] GraySentinel Azure AD Audit Starting...
[+] Running ROADrecon...
[+] Gathering Azure AD data...
[+] Generating report...
[+] Running AzureHound...
[+] Audit Complete! Reports saved in graysentinel-azure-audit-20260822
| 💡 Executes the full Azure AD audit script.
```

Evidence 11: Consolidated report generation and remediation table

```

azure@kali:~$ python3 graysentinel_azure_report.py
[+] Generating GraySentinel Azure AD Security Report...
[+] Parsing ROADrecon findings...
[+] Parsing AzureHound data...
[+] Building report...
✅ Report generated: graysentinel-azure-report.html
| 💡 Generates a consolidated HTML report from all findings.

azure@kali:~$ cat > azure_ad_misconfigurations.md <<EOF
# Azure AD Misconfigurations & Attack Paths
| # | Misconfiguration | Detection Method | Severity | Attack Path | Remediation |
|---|-----|-----|-----|-----|-----|
| 1 | Global Admin with No MFA | ROADrecon | 🔴 Critical | Password spray → Global Admin | Enable MFA |
| 2 | Service Principal High Privileges | ROADrecon | 🔴 Critical | SP → Azure Resource access | Least privilege |
| 3 | App Permissions Over-granted | AzureHound | 🟡 High | App → Graph API → Data exfiltration | Review permissions |
| 4 | Guest Users with Admin Roles | ROADrecon | 🟡 High | Guest → Privilege escalation | Remove guest admins |
| 5 | Weak Conditional Access | ROADrecon | 🟡 Medium | Policy bypass → Unauthorized access | Require MFA for all |
| 6 | Legacy Authentication Enabled | ROADrecon | 🟡 High | Legacy auth → Password spray | Disable legacy auth |

```

Evidence 12: Achievement checklist and audit-directory validation

```

azure@kali:~$ cat > achievement_checklist.md <<EOF
# Day 13 – Azure AD & Entra ID Attacks ✅
| Task | Status | Tool |
|-----|-----|-----|
| Stormspotter Install & Run | ✅ Done | docker, git |
| AzureHound Install & Data Collection | ✅ Done | azurehound |
| ROADtools Install & Enumeration | ✅ Done | roadrecon |
| MSOLSpray Password Spraying | ✅ Done | msolspray |
| Attack Path Mapped | ✅ Done | Stormspotter + BloodHound |
| 3+ Misconfigurations Identified | ✅ Done | Manual |
| Report Generated | ✅ Done | HTML/JSON |
EOF
| 💡 Tracks your progress – check off each completed task.

azure@kali:~$ ls -la graysentinel-azure-audit-*
graysentinel-azure-audit-20260822:
total 128
drwxr-xr-x 2 azure azure 4096 Aug 22 14:30 .

```

```
azure@kali:~$ ls -la graysentinel-azure-audit-*
graysentinel-azure-audit-20260822:
total 128
drwxr-xr-x 2 azure azure 4096 Aug 22 14:30 .
drwxr-xr-x 4 azure azure 4096 Aug 22 14:30 ..
-rw-r--r-- 1 azure azure 12345 Aug 22 14:30 roadrecon-users.json
-rw-r--r-- 1 azure azure 9876 Aug 22 14:30 roadrecon-groups.json
-rw-r--r-- 1 azure azure 23456 Aug 22 14:30 roadrecon-sp.json
-rw-r--r-- 1 azure azure 15678 Aug 22 14:30 azurehound-data.json
-rw-r--r-- 1 azure azure 2345 Aug 22 14:30 msolspray-results.txt
-rw-r--r-- 1 azure azure 5678 Aug 22 14:30 graysentinel-azure-report.html
| 📢 Lists the contents of the audit directory – shows all generated reports.
azure@kali:~$ echo "Rank: Cyber Commando (Azure)"
🏆 Rank: Cyber Commando – Azure AD Security Specialist
| 📢 5. Final Thoughts & Next Steps: Securing Your Azure AD Environment
```

Appendix B - Public GitHub Publishing Checklist

- Remove plaintext passwords, client secrets, tokens and authentication caches.
- Replace real tenant domains, emails and IDs with placeholders unless intentionally public.
- Do not commit .env files, shell history, credential caches or unredacted evidence.
- Add a .gitignore for sensitive/local artifacts.
- Keep only sanitized commands and screenshots in the public repository.
- Include a clear authorized-testing disclaimer.